Day 03 – Git Branches & Branch Pointers

Topics Covered

Git Branches
Why branches are lightweight
Branch pointers
HEAD movement
git branch / git switch / git checkout
.git/refs/heads
Branch deletion (-d vs -D)
Git Flow, GitHub Flow, Trunk-Based Development

Git Branch Architecture

Before: HEAD → main → Commit C

After creating feature:
HEAD → main → Commit C
feature → Commit C

After switching:
HEAD → feature → Commit C
main → Commit C

After committing:
HEAD → feature → Commit D → Commit C
main → Commit C

Commands

git branch
git branch feature
git switch feature
git switch main
git checkout feature
git checkout main
git branch -d feature
git branch -D feature
ls .git/refs/heads
cat .git/refs/heads/main

Production Notes

A branch is a lightweight movable pointer to a commit.
Git does not copy the repository when creating a branch.
Local branches are stored in .git/refs/heads/.
Each branch file stores the SHA-1 hash of the commit it points to.
Only the active branch pointer moves after a commit.

Interview Questions

1. What is a Git branch?
2. Why are Git branches lightweight?
3. Where are branches stored?
4. What does .git/refs/heads/main contain?

5. What happens during git checkout?
6. What moves after a commit?
7. Difference between -d and -D?
8. Explain Trunk-Based Development.

Day 03 Outcome

You can now explain Git branches, HEAD, branch pointers, branch storage, branch switching, deletion, and modern branching strategies.